

PaddlePaddle 学习笔记

一、背景

本文档记录了本人在入职学习 PaddlePaddle 代码过程中，学习到的一些知识点。这多种表达。如 `proto::BlockDesc`、`BlockDesc`、`Block` 等。不都是 `Block` 吗，为啥还分出来好几种呢？

Protocol Buffer 是，是 Google 开发的一种数据描述语言，在数据结构序列化、数据存储、数据通信方面功能强大。Paddle 使用 Proto 定义 Program、Block、Var、OP 等数据结构，有利于数据的存储、共享等。Proto 数据结构定义在 .proto 文件中，Google 提供了 protoc 编译器，通过该编译器，可将 .proto 『编译』成 C++、Python 等代码。Paddle 代码中『`proto::xxxDesc`』字样得代码即 Protocol Buffer 『编译』产生的。

为了更好的在框架中使用 `proto::xxxDesc`，我们对 `proto::xxxDesc` 进行了些知识点主要来自三方面：

- 1) 同事口头讲解；
- 2) 部门内现有文档；
- 3) 自己通过阅读代码、思考总结。

PaddlePaddle 的代码是复杂的，入门周期较长。目前项目已经过多轮迭代，但在代码中无法 Get 到『历史渊源』。而从代码反映『历史渊源』，进而看懂代码逻辑是很吃力的。因此，这个笔记，主要用文字表述一些『渊源』、『背景』、『目的』，总结一些『程序执行流程』。希望对大家有帮助。里面有些逻辑也很难使用文字语言表达清楚，所以有些地方表述欠佳。很多时候本文档，通过文字总结程序的执行过程，类似于代码注释，但必须结合代码阅读才行，否则读不懂。

此外，目前（编写文档时）本人对 Paddle 的理解还有限。里面可能有一些猜测、结论是错误的，有可能误导阅读者。

本文假设阅读的同学对深度学习有简单的了解。如果需要补充深度学习知识，可以看看百度 AI Studio 中的教程：

<https://aistudio.baidu.com/aistudio/course>

还有一个特别好的视频链接如下：

<https://space.bilibili.com/88461692/channel/detail?cid=26587>

二、我对 PaddlePaddle 的理解

Paddle 设计之初，希望使用 Paddle 编写深度学习模型的用户，就像正常编写程序一样『表述』模型。因此，有别于其它深度学习框架，创新的提出了 Program 的概念。而业界深度模型更多采用 Graph 的概念。Program 最大特点是支持了类似与 If、While 等条件控制类语法，让 Paddle 对模型的表达能力更加强大。（以上观点非官方，纯属个人见解）

既然，支持了条件控制类语法，Paddle 就引入了 Block 的概念，类似于 C++ 中的 {}，代表程序块、作用域等。每个 Block 中有大量的 Operator、Variable，即算子和数据。这些算子和数据组成了一个复杂的深度学习网络模型。

范范的来说，Paddle 框架的主要任务有：

- 1) 提供友好的 API 接口，让用户『表达』、『开发』模型更容易。Paddle 将其转化为 Program+Block+Operator+Variable 的程序表达；
- 2) 对『Program+Block+Operator+Variable 的程序表达』进行必要、且尽可能充分的优化，以使得模型训练高效；
- 3) 业界有大量成熟的网络模型（fc、conv2d 等），Paddle 应对这些模型进行封装，用户拿去用就可以了。
- 4) fc、conv2d 等，可以理解为由无数个 Operator 组成的，一个 Operator 完成一个小的数学运算。因此 Paddle 需要开发大量的 Operator，并有效的管理 Operator。同时保证很好的 Operator 扩展性。sc、VarDesc 的计算机的存储结构，这种存储结构的信息是充分持有用户所表达的信息的。
 - 1) 用户提前准备好需要训练的数据。
 - 2) 用户将定义好的 Program 和准备好的数据，交给 Executor 去 Run。

3.1 基础数据结构、概念

3.1.1 Variable 和 Operator

深度学习网络模型，本质上都是在进行数据运算，无论多复杂的数学公式，多大的模型，最终都可以拆解成一个一个的小的计算单元和数据存储单元。Operator 就是最基础的计算单元，Variable 就是最基础的数据存储单元。

因此 Variable 和 Operator 是一个网络模型的最小组成元素，任何模型均由无数个 Variable 和 Operator 组成。

- 5) Variable 和 Operator 是 Paddle 底层的核心最核心的概念。几乎所有逻辑都是围绕这两个概念展开。
- 6) 用户定义好模型后，Paddle 应负责模型的训练。Paddle 应尽可能多的支持各种硬件，如 CPU、GPU、ARM、FPGA 等；
- 7) 在训练过程中，为了提升性能、加快训练，Paddle 需要考虑程序优化、内存优化，也需要考虑并发、多卡、分布式；
- 8) 训练完得模型，应该可存储、可加载，可预测。

因此，路漫漫其修远兮。。。

三、静态图+Executor

静态图+Executor 是 Paddle 最早支持的方案。

从用户视角，其大致使用流程为：

- 3) 用户调用 Python (Paddle API) 代码定义网络模型。此时, Paddle 将网络模型转义成 (ProgramDesc、BlockDesc、OPDe

3.1.2 Program 和 Block

Program 和 Block 是 Paddle 提出的, 对 OP (Operator) 和 Var (Variable) 的管理方法、概念。

Program 可以理解为一段程序, 他描述了一个模型的整体长相。

而程序中可能会有 if、while 等程序块, 每个『块』, 即一个 Block。

因此一个 Program 中可能有 1 个或多个 Block, 其中首个 Block 存储的是 Program 中的 OP、Var 集合, 剩余 Block 分别存储相应的子程序块中的 OP、Var 集合。

Program 可以交付给 Executor 执行, Executor 首先执行 Block[0]中的 OP, 过程中根据情况穿插执行其余 Block。

3.1.3 Proto 和 Desc

在 Paddle 中 Program、Block、Variable、Operator, 每个都有一层包装, 即我们的 C++表示——xxxDesc, 如 VarDesc, OpDesc。

3.1.4 Python 中的 Program、Block、Variable、Operator

Python 端主要负责 Program 的构建, 我们叫『编译时』。为了 Python 端构建 Program 时, 方便使用 ProgramDesc、BlockDesc、VarDesc、OpDesc。框架在 Python 端定义了与之对应的 Program、Block、Variable、Operator 四个 Python 类, 每个类均有一个成员变量 self.desc, self.desc 持有 C++的 xxxDesc 对象。这样, Python 随时可以调用 C++中的方法。

3.1.5 Parameter

模型中的可学习的参数, 我们称为 Parameter。Parameter 是一种特殊的 Variable。我们训练的目的就是找到最合理的 Parameter 集合, 能够让网络得出最好的预测结果。

3.1.6 C++中的 Variable、OperatorBase

以上所有的 Program、Block、Variable、Operator, 都属于对 Program 的『描述』, 但都不是可执行的计算机表达。所有的『描述』都是为了能够快速构建、和优化 Program。当 Program 交给 Executor 去执行时, Executor 必须把 Program 中的『描述』信息, 转化为真正可执行计算的 Operator 和可存储变量数据的 Variable。这就是 C++中的 Variable 和 OperatorBase。

为了支持不同种类的数据, 因此 Variable 是动态模板类。Variable 代表可被算子计算使用的变量, 即内存数据。Variable 通过封装, 内部可以存储多重类型的变量。如下图所示 (来自 var_type_traits.h) :

```

REG_PROTO_VAR_TYPE_TRAIT(LoDTensor, proto::VarType::LOD_TENSOR);
REG_PROTO_VAR_TYPE_TRAIT(SelectedRows, proto::VarType::SELECTED_ROWS);
REG_PROTO_VAR_TYPE_TRAIT(std::vector<Scope *>, proto::VarType::STEP_SCOPES);
REG_PROTO_VAR_TYPE_TRAIT(LoDRankTable, proto::VarType::LOD_RANK_TABLE);
REG_PROTO_VAR_TYPE_TRAIT(LoDTensorArray, proto::VarType::LOD_TENSOR_ARRAY);
REG_PROTO_VAR_TYPE_TRAIT(platform::PlaceList, proto::VarType::PLACE_LIST);
REG_PROTO_VAR_TYPE_TRAIT(ReaderHolder, proto::VarType::READER);
REG_PROTO_VAR_TYPE_TRAIT(FetchList, proto::VarType::FETCH_LIST);
REG_PROTO_VAR_TYPE_TRAIT(int, proto::VarType::INT32);
REG_PROTO_VAR_TYPE_TRAIT(float, proto::VarType::FP32);

```

为了支持各类 OP，OperatorBase 只是个几类，定义了关键接口函数。OP 数量很多，Paddle 提供了较好的注册、使用机制。具体可学习胡晓光的《PaddlePaddle 深入浅出》和陈威行的《Paddle Fluid 框架执行逻辑梳理》。

3.1.7 C++中的 Scope

Scope 是运行时概念，主要用于训练过程中 Var 的管理。在 Executor 运行过程中，将某些 Var 指针放到同一个 Scope 中，这些 Var 可能具有相同的生命周期、或者相同的属性，放到一起方便管理。

3.1.8 Place

Paddle 支持在多种计算机硬件进行模型训练。Place 用于描述，训练所用的硬件。如 CPU、GPU、ARM、FPGA 等等。

Executor 在实例化 C++ OperatorBase、Variable 时，会根据 Place 的不同，实例化对应的 OPKernel 负责计算。也会根据 Place，将 Variable 用于存储数据的内存，分配到对应的 Place 中去。

3.1.9 Executor

概念上 Executor 是执行器，负责 Program 的执行。用户通过 Python 接口定义 Executor 实例，传入 Program 进行 Program 的执行。

Python 端的 Executor 是编译时和运行时的过度。他完成了 Program 最后的一些构造和优化，完成了 C++ Executor 的创建，启动了 C++ Executor 的 Run。Python 的 Executor 负责整个 Program 的启动执行，因此，他传入 C++ Executor Run 的 BlockID 为 0。因为 Block0 为主 Block。

C++的 Executor 可以说是真正的执行器。他拿到 Program 的 Desc，将其转化为 C++可执行的 Variable 和 OperatorBase，并完成执行。

我们通常说的 Executor 指的是 C++的 Executor，或者指整个静态图+Executor。这需要我们结合语境判断。

3.2 整体流程概览

Executor 可分为编译时和运行时，两个阶段。编译时主要由 Python 完成，主要负责 Desc（Desc 指的是 ProgramDesc+BlockDesc+OpDesc+VarDesc）的构造和优

3.3 编译时

编译时的任务是生成 Program。他的输入可以认为是用户调用的 API，用户通过调用 API 定义网络模型。他的输出是 Program，Program 中拥有用户所描述的网络模型的全部信息。下页图片是一个 Program 的图片表达，3.2 的 Demo 可以认为是编译时的输入，下页图片可以认为是编译时的输出。

图片见下页

Program 由多个 Block 组成使用 Vector 存储。Block 中有 OP 和 Var。OP 执行是有先后循序的，因此 OP 使用 deque 存储。OpDesc 中定义了 inputs_、outputs_，里面包含了 Var 的 Name。

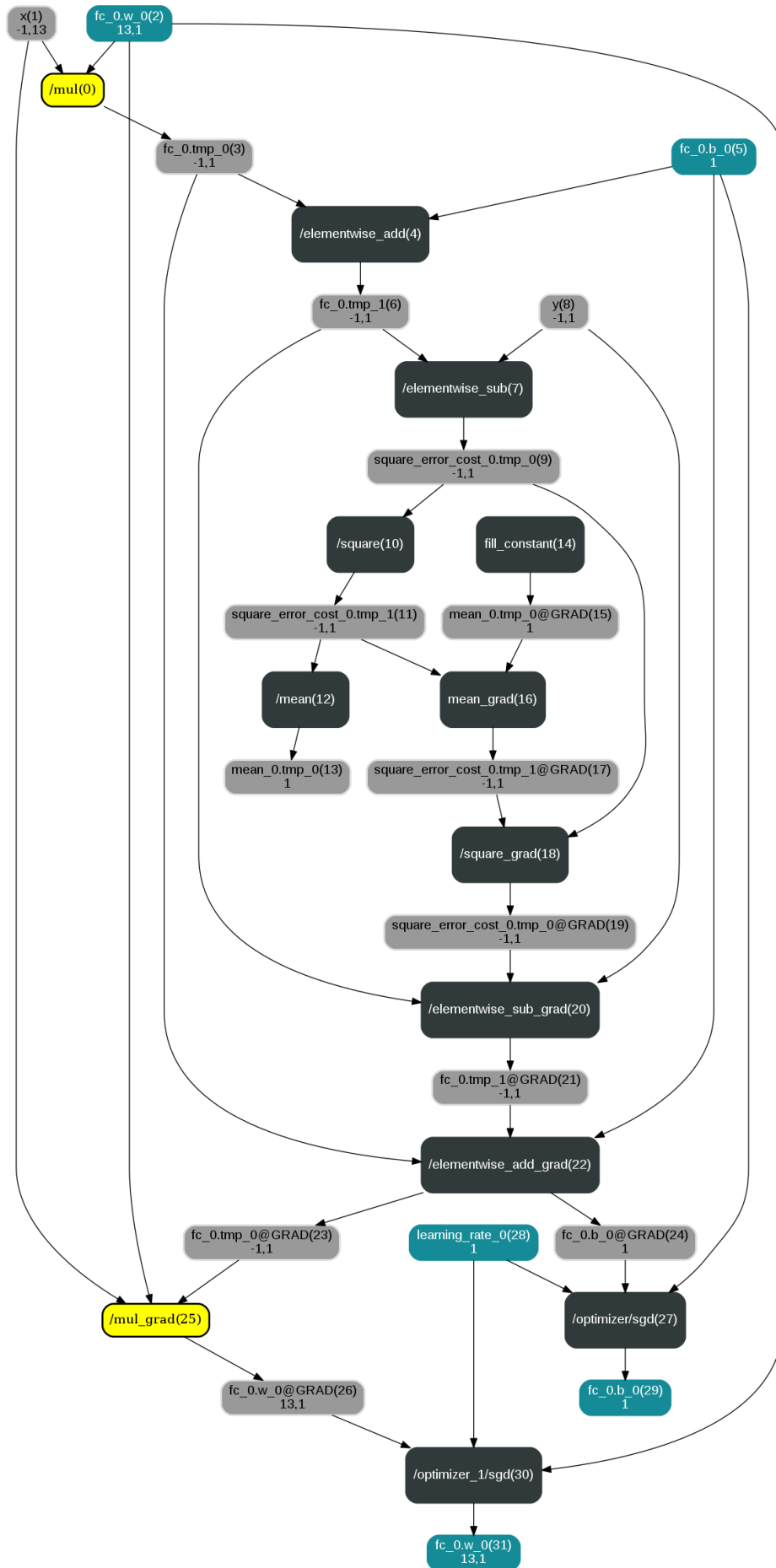
通过 OpDesc 中的 inputs_、outputs_，我们隐晦的表达了一个图。这个图的节点是 Var 和 Op，边是输入输出的依赖关系。这是个真正的图，因为它拥有图的所有必要信息，但是在数据结构存储中，表达的是不直接的，因此说它隐晦。

结论，编译时，产出的是能够充分表达深度学习网络定义的 Program。Program 由 Block 构成，Block 由 OpDesc 和 VarDesc 构成。Block 的本质是以 OpDesc、VarDesc 为节点，以输入输出为方向的图。但在 Block 中存储的形式，让这个图的表达比较隐晦。

从下图中，我们看到了几类节点：

- 1) 我们自己定义的 X、Y
- 2) FC 网络所必要的 OP，如 mul、elementwise_add、relu
- 3) FC 网络计算所产生的一些临时中间 Var，如 fc_0.tmp0
- 4) FC 网络的参数 (Parameter)，如 fc_0.w_0, fc_0.b_0
- 5) 计算均方差所产生必要 OP 和 Var
- 6) 每个前向 OP，所对应的，计算反向下降梯度的 OP_grad
- 7) 每个前向 Var、Param 的反向 Var_grad。注意：不是所有的 Var 都需要计算反向 Var_grad，比如我们输入的 X、Y 计算反向 grad 就没有意义
- 8) 最终优化参数的 sgd_op

其中 1~5 是前向产生的，根据用户的网络定义生成。6~8 是 Paddle 根据 1~5 的信息，由 Paddle 编译时自动补全的。



3.3.1 前向

3.3.1.1 占位 Variable

```
13 x = fluid.layers.data(name='x', shape=[13], dtype='float32') # 前向—占位Variable
14 y = fluid.layers.data(name='y', shape=[1], dtype='float32') # 前向—占位Variable
15 print(fluid.default_main_program())
```

DEBUG CONSOLE PROBLEMS 132 TERMINAL OUTPUT

```
grep: warning: GREP_OPTIONS is deprecated; please use an alias or script
{ // block 0
  var x : fluid.VarType.LOD_TENSOR.shape(-1, 13).astype(VarType.FP32)
  var y : fluid.VarType.LOD_TENSOR.shape(-1, 1).astype(VarType.FP32)
}
```

通过上图，我们可以看出，当执行完两个占位 Variable API 后，Program 的 Block0 中多了两个 Var，分别为 x、y，并记录了 x、y 的 shape、类型等。

其创建过程大致如下：

- 1) 因为这个 Var 是用户在全局声明的，因此，调用 GlobalBlock（即 Program 的 Block0）创建。
- 2) 创建 Var 即实例化了一个 Python 的 Variable。Variable 构造函数，通过调用 C++ 的 BlockDesc，创建一个 C++ 的 VarDesc，并在 Python 的 Variable 中持有一份 (self.desc)。以下是 Variable 构造函数的部分代码截图：

```
if self.desc is None:
    self.desc = self.block.desc.var(cpt.to_bytes(name))
    is_new_var = True
```

3.3.1.2 FC 网络定义

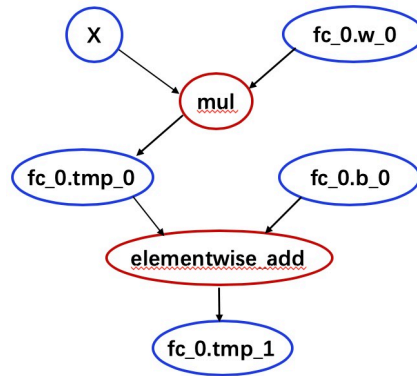
```
13 x = fluid.layers.data(name='x', shape=[13], dtype='float32') # 前向—占位Variable
14 y = fluid.layers.data(name='y', shape=[1], dtype='float32') # 前向—占位Variable
15 #print(fluid.default_main_program())
16
17 y_predict = fluid.layers.fc(input=x, size=1, act=None) # 前向—网络定义，计算OP、中间Var、相关Param参数
18 print(fluid.default_main_program())
```

DEBUG CONSOLE PROBLEMS 136 TERMINAL OUTPUT

```
{ // block 0
  var x : fluid.VarType.LOD_TENSOR.shape(-1, 13).astype(VarType.FP32)
  var y : fluid.VarType.LOD_TENSOR.shape(-1, 1).astype(VarType.FP32)
  persist trainable param fc_0.w_0 : fluid.VarType.LOD_TENSOR.shape(13, 1).astype(VarType.FP32)
  var fc_0.tmp_0 : fluid.VarType.LOD_TENSOR.shape(-1, 1).astype(VarType.FP32)
  persist trainable param fc_0.b_0 : fluid.VarType.LOD_TENSOR.shape(1, 1).astype(VarType.FP32)
  var fc_0.tmp_1 : fluid.VarType.LOD_TENSOR.shape(-1, 1).astype(VarType.FP32)

  {Out=['fc_0.tmp_0']} = mul(inputs={X=['x'], Y=['fc_0.w_0']}, force_fp32_output = False, op_device = , op_namescope = /, scale_x = 1.0, scale_y = [1.0], use_mkldnn = False, x_num_col_dims = 1, y_num_col_dims = 1)
  {Out=['fc_0.tmp_1']} = elementwise_add(inputs={X=['fc_0.tmp_0'], Y=['fc_0.b_0']}, axis = 1, op_device = , op_namescope = ,
  else, x_data_format = , y_data_format = )
}
```

通过上图，我们看到 fc 函数，执行后，Program 中多了 fc_0.tmp_0，fc_0.tmp_1 两个普通 Var，多了 fc_0.w_0，fc_0.b_0 两个 Parameter 和 mul、elementwise_add 两个 OP，信息非常丰富（足够表达清楚网络模型需要的计算行为）。他们之间的关系如下：



总结：像 `fc` 这样的 Python 函数，他的任务就是构造 OP、Var 之间的关系，并把 Desc 存入 Program 中。

题外话：这点映射出了 *Paddle* 中底层的大量工作：

- 1) 我们需要开发大量类似于 FC 的网络模型，业界有的我们都要支持，如 `fc`、`conv2d`；
- 2) 每个网络模型都由 N 个 OP 组成，因此我们要开发更大量的 OP；
- 3) OP 要能够在不同的硬件（CPU、GPU 等）运行，不同的硬件、不同的变量类型，OP 的实现方式是不一样的。因此 OP 的开发是大量大量的！

3.3.1.2.1 临时变量 Var 的构造

这里说的临时变量 Var 其实就是 OP 与 OP 之间传递数据的临时变量。FC 创建 Var 过程与占位 Variable 基本一致，区别在于占位 Variable 的 `stop_gradient` 为 `True`，而 FC 中的 Var 的 `stop_gradient` 为 `False`。

为啥 `stop_gradient` 不同呢？读完后文，这个疑问自会消失。

3.3.1.2.2 Parameter Var 的构造

Parameter 的父类是 Variable，但是 Parameter 的创建与 Variable 有些不同：

- 1) Parameter 的 `persistable` 为 `True`，代表不能被优化掉
- 2) Parameter 的 Create，需要让 `StartupProgram` 和 `MainProgram` 都 Create 一份
- 3) Parameter 的 `name` 通常为 `w`、`b` 字样

3.3.1.2.3 FC 中 OP 的创建

FC 创建 OP 过程大致如下：

- 1) FC 提供 OP 创建的基本信息，包括输入输出变量，OP 类型，OP 的 Attr
- 2) 通过调用 C++ `BlockDesc` 的 `append_op`，获得一个创建好的 C++ `OpDesc`
- 3) 实例化 Python 端的 Operator，在 Operator 的构造函数中，完善 C++ `OpDesc` 中所需的信息

总结：编译时，创建 Var、OP 的本质是构造 C++ 的 VarDesc、OpDesc，并通过与 Python 的对象关联，实现 Python 端对 Desc 的管理，进而实现 Program 的构造。

题外知识点：

- *StartupProgram*

这里需要展开介绍一下 StartupProgram。正常训练模型有两个 Program，一个 StartupProgram 和一个 MainProgram。StartupProgram 负责 Parameter 的初始化，MainProgram 负责训练模型的执行。

问题 1：为啥不在 MainProgram 中加几个初始化的 OP 完成 Parameter 的初始化呢？因为 MainProgram 需要循环跑非常多轮，而 Parameter 只需要初始化一次。放在 MainProgram 中需要对初始化 Parameter OP 进行特殊处理，只允许第一次 Run 时执行。

问题 2：StartupProgram 怎么初始化的？create_parameter 时，给 StartupProgram 传入了 with_initializer=True 的参数，用于标记需要插入初始化 OP。因此 StartupProgram 在创建 Var 的同时，还向 Block 中插入相应的初始化 OP，如 fill_constant、uniform_random。用户在开发时，在 Run MainProgram 前，只要先 Run StartupProgram 就可以完成 Parameter 的初始化。详见 create_parameter 代码。

问题 3：StartupProgram 初始化 Parameter，MainProgram 就可以共享使用吗？在编译时，StartupProgram 和 MainProgram 分别 Create 了一个 VarDesc，也就是 VarDesc 由 2 份。但是在 Executor Run 两个 Program 时，传入的 Scope 都是 GlobalScope。因此在 Run StartupProgram 时，在 GlobalScope 中已经实例化了 C++ 的 Variable，并分配了内存，初始化了数据。到 Run MainProgram 时，就可以从 GlobalScope 中 find 到了，不会二次实例化 Variable。详见 Python Executor Run 代码。

- *stop_gradient & persistable*

stop_gradient 是 Python 端 Variable 的成员变量，在 C++ 里没有。主要用于反向 minimize 使用，因此只有 Python 端才用的到。

stop_gradient 标记了一个 Var 是否需要计算反向的 grad，即梯度下降值。占位 Var 是外部 Feed 进来的，计算 grad 是无意义的。而 FC 中的临时变量，可以向上传导参数的修改期望，因此需要计算 grad。

因为 Feed、Fetch Var 是在 minimize（反向 OP、反向 Var 添加）之后添加的，因此不关心 stop_gradient。

stop_gradient 主要用于标记前向 Var，因此 grad Var 不需要标记。而且 grad Var 只创建了 C++ VarDesc，没有创建 Python Variable，因此不涉及 stop_gradient。

persistable，是用于描述这个 Var 是否可以被优化掉，一般 Feed Var、Fetch Var、Parameter 是不允许被优化的。Persistable 是在 proto 中就定义好的一个属性（Var 有三个属性：name、type、persistable），因此与 stop_gradient 不同，persistable 属性需要同步到 C++ 的 VarDesc 中。

以下做个总结（可能有些特殊情况下，这个总结是错的，但大多数情况下如此）

	Feed/Fetch Var	占位 Var	Parameter	tmp Var	grad Var
--	----------------	--------	-----------	---------	----------

<i>stop_gradient</i>	<i>None</i>	<i>TRUE</i>	<i>TRUE</i>	<i>FALSE</i>	<i>无</i>
<i>persistable</i>	<i>TRUE</i>	<i>FALSE</i>	<i>TRUE</i>	<i>FALSE</i>	<i>FALSE</i>

3.3.1.3 均方差损失函数

均方差，可以代表预测值与实际值的偏差。按计算先后顺序为差、方、均。这部分代码很简单，每个 OP 之后的输出均需要一个临时变量。以 `square_error_cost` 为例：

```
helper = LayerHelper('square_error_cost', **locals())
minus_out = helper.create_variable_for_type_inference(dtype=input.dtype)
helper.append_op([
    type='elementwise_sub',
    inputs={'X': [input],
            'Y': [label]},
    outputs={'Out': [minus_out]})

square_out = helper.create_variable_for_type_inference(dtype=input.dtype)
helper.append_op(
    type='square', inputs={'X': [minus_out]},
    outputs={'Out': [square_out]})
```

3.3.1.4 控制流

3.2 节中的 Demo 为顺序执行。如果我们希望在定义网络时通过 IF、WHILE 表述一些复杂的模型，则需要用到控制流。

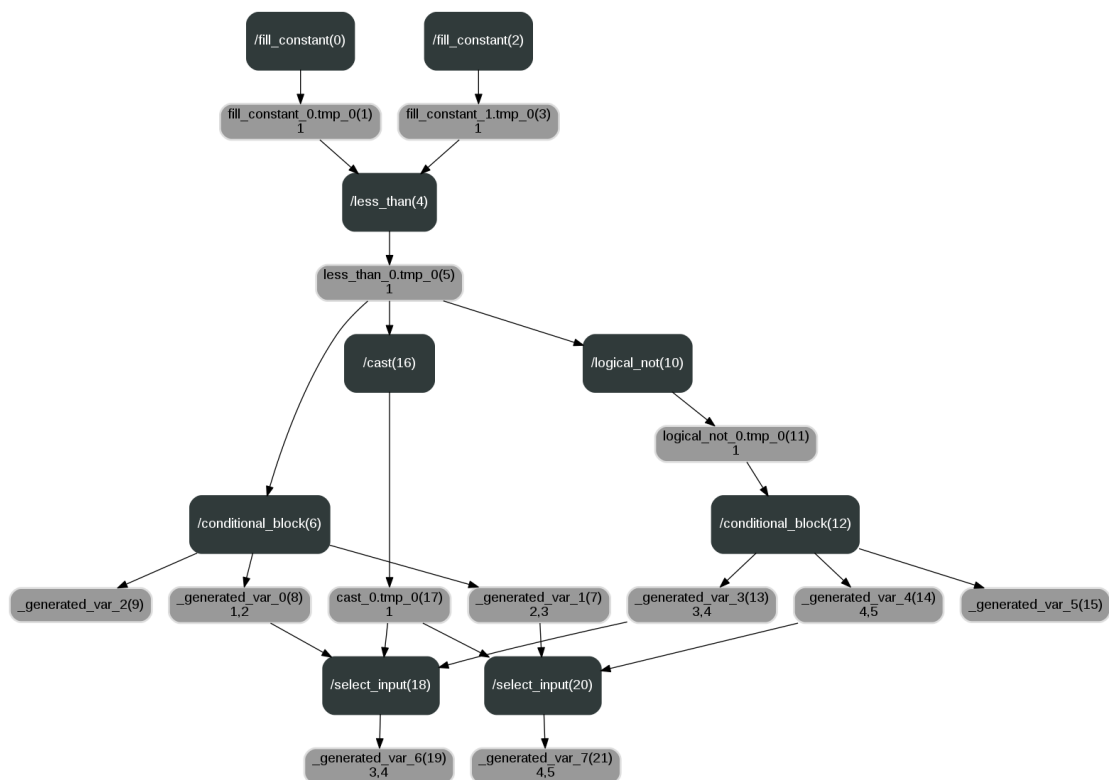
我们以 Paddle 官网 `cond` 的 Demo 为例，介绍控制流的编译时是如何构造 Program 的。代码如下：

```
def true_func():
    return layers.fill_constant(
        shape=[1, 2], dtype='int32', value=1), layers.fill_constant(
            shape=[2, 3], dtype='bool', value=True)

def false_func():
    return layers.fill_constant(
        shape=[3, 4], dtype='float32', value=3), layers.fill_constant(
            shape=[4, 5], dtype='int64', value=2)

main_program = Program()
startup_program = Program()
with program_guard(main_program, startup_program):
    x = layers.fill_constant(shape=[1], dtype='float32', value=0.1)
    y = layers.fill_constant(shape=[1], dtype='float32', value=0.23)
    pred = layers.less_than(x, y)
    out = layers.cond(pred, true_func, false_func)
```

首先我们看一下这段代码所生成的 Program 的 Block0 的 Graph 打印：



除了 Block0 外，编译时还会为 true_func 和 false_func 中的网络定义会分别创建一个新的子 Block（Block1 和 Block2），分别描述两个分支，信息如下：

```

{ // block 1
  var fill_constant_2.tmp_0 : fluid.VarType.LOD_TENSOR.shape(1, 2).astype(VarType.INT32)
  var fill_constant_3.tmp_0 : fluid.VarType.LOD_TENSOR.shape(2, 3).astype(VarType.BOOL)

  {Out=['fill_constant_2.tmp_0']} = fill_constant(inputs=[ShapeTensor=[]], ValueTensor=[], dtype = 2, force_cpu = False, op_device = , op_namescope = /
  , op_role = 0, op_role_var = [], shape = [1, 2], str_value = 1, value = 1.0)
  {Out=['fill_constant_3.tmp_0']} = fill_constant(inputs=[ShapeTensor=[]], ValueTensor=[], dtype = 0, force_cpu = False, op_device = , op_namescope = /
  , op_role = 0, op_role_var = [], shape = [2, 3], str_value = 1.0, value = 1.0)
  {Out=['_generated_var_0']} = assign(inputs=(X=['fill_constant_2.tmp_0']), op_device = , op_namescope = /, op_role = 0, op_role_var = [])
  {Out=['_generated_var_1']} = assign(inputs=(X=['fill_constant_3.tmp_0']), op_device = , op_namescope = /, op_role = 0, op_role_var = [])
}
{ // block 2
  var fill_constant_4.tmp_0 : fluid.VarType.LOD_TENSOR.shape(3, 4).astype(VarType.FP32)
  var fill_constant_5.tmp_0 : fluid.VarType.LOD_TENSOR.shape(4, 5).astype(VarType.INT64)

  {Out=['fill_constant_4.tmp_0']} = fill_constant(inputs=[ShapeTensor=[]], ValueTensor=[], dtype = 5, force_cpu = False, op_device = , op_namescope = /
  , op_role = 0, op_role_var = [], shape = [3, 4], str_value = 3.0, value = 3.0)
  {Out=['fill_constant_5.tmp_0']} = fill_constant(inputs=[ShapeTensor=[]], ValueTensor=[], dtype = 3, force_cpu = False, op_device = , op_namescope = /
  , op_role = 0, op_role_var = [], shape = [4, 5], str_value = 2, value = 2.0)
  {Out=['_generated_var_3']} = assign(inputs=(X=['fill_constant_4.tmp_0']), op_device = , op_namescope = /, op_role = 0, op_role_var = [])
  {Out=['_generated_var_4']} = assign(inputs=(X=['fill_constant_5.tmp_0']), op_device = , op_namescope = /, op_role = 0, op_role_var = [])
}

```

对于两个子 Block 的创建，请看下面代码中我添加的注释：

```

true_cond_block = ConditionalBlock([pred], is_scalar_condition=True) #创建一个新的Block, 并在with结束时 (complete) 创建一个CondBlockOP
with true_cond_block.block(): #选择了新创建的Block
    origin_true_output = true_fn() #在新创建的Block中执行true_func, 而true_func中由两个constantOP, 返回两个参数, 这些Var、OP都会Append到新的Block中
    if origin_true_output is not None:
        true_output = map_structure(copy_to_parent_func,
                                    origin_true_output) #添加在父Block中创建接受cond返回值的参数, 创建assign (复制) OP, 将数据复制到父Block中

```

cond 编译时构造 Program 的大致流程如下（详见 python cond 函数代码）：

- 1、创建两个子 Block，分别描述两个分支，完成两个子 Block OP、Var 的 Append
- 2、两个分支不可能都执行，到底执行哪一个？需要 Block0 中通过 CondBlockOP 决定。在 Block0（父 Block）中，创建两个 CondBlockOP，每个 OP 分别用于决定其负责的 Block 是否被执行。其输入分别是 pred 和 logical_not(pred)，因此也引入了 logical_not 的一个 OP。那么哪个 CondBlockOP 负责哪个 Block 的执行呢？通过在 CondBlockOP 的 Attr 中插入 sub_block，将整个 BlockDesc

都存进去。CondBlockOP Run 的时候，从去获得 blockid 就知道其对应的 Block 是哪个了。

3、将 bool 类型的 pred 转成 int32 类型的 mask（就是 castop）这部不知道处于啥原因。

4、因为要返回两个 Var，所以用两个 select_input_op 做选择，输出给外部。

3.3.2 后向

用户定义好前向网络后，编译时逻辑已将所有的 OP、Var 生成好，append 到相应的 Block 中，而反向 OP 和相应的 Var 则由编译时逻辑自动生成。

3.3.2.1 优化器选择

优化器，本质是深度学习的优化算法。Paddle 支持常见的优化算法，如 SGD、Adam、Adamax、Momentum、Adagrad 等等。

这些优化算法分别由一个 Python 的类实现。如 SGDOptimizer、AdamOptimizer。他们都集成自 Optimizer 类。而绝大部分编译时工作，其实是由 Optimizer 完成的，而子类主要是通过重写 _append_optimize_op 函数，为 Program 添加 optimize_op（就是优化算法的实现）。optimize_op 是在 minimize 最后插入的。

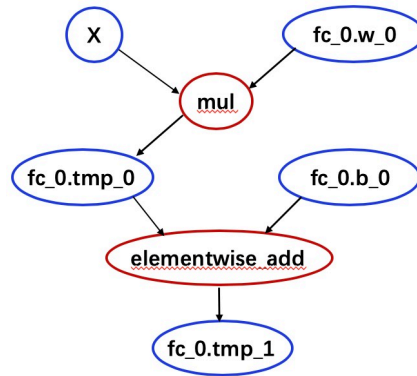
Optimizer 的核心函数是 minimize，该函数负责反向 OP 的生成。

3.3.2.2 minimize

minimize 负责 OP_grad、Var@grad、Param@grad、Optimizer_OP 的生成。

minimize 过程很是复杂，本人没能彻底看懂，大致流程如下：

- 1) 首先为每个 Block 构造 no_grad_set，对于 Block0，包括用户给的、not trainable 的 Parameter 和 stop_grad 为 True 的 Variable。其它 Block 的 no_grad_set 为 stop_grad 为 True 的 Variable。
- 2) 为每个 Block Append OP_grad 和 Var@grad，其中 Var@grad 中也包括了 Param@grad。Block0，直接 Append 到自己的 Block 中。ctrl_flow Block，为其创建新的、对应的 Block，往新的 Block 中 Append；
 - a) 遍历本 Block 中所有的 OP，找到一个 op_path，op_path 是所有 OP 的一个子集，优化掉了部分不需要创建 grad 的 op。
 - b) 反向遍历 op_path 中所有 OP，调用 C++ 的 GradOpMaker，为该 OP 创建 OP_grad。
 - c) 创建的 OP_grad，并不是都有用。结合 input、output、no_grad_set，把不必要的路径做剪枝。打个比方：假如 X 是 stop_grad 的，若 OPx 只生成了 X 一个变量，那么 OPx 没必要创建 grad。同时，OPx 之上的部分 Var、OP 也可以做剪枝（有点儿抽象不好理解）。以下图为例，假如 fc_0.tmp_1 为 stop_grad，那么 elementwise_add 就不需要 OP_grad 了，而之上的所有枝叶，均能被剪枝。



- d) 把剩余的 OP_grad Append 到对应的 Block 中去。
- 3) 2) 的过程收集了大量的 grad_to_var, 需要 rename 保证名字不重复。
- 4) 将所有的 Var@Grad Append 到对应的 Block 中去。
- 5) 根据 Parameter_list 和 grad_info_map, 生成一个清单。里面包含需要 grad 的 Parameter 和 Param@Grad。
- 6) 调用每个优化器重载的 _append_optimize_op 函数, 根据 Parameter 和 Param@Grad, 生成对应的几个 Optimizer_OP。

minimize 计算反向过程比较复杂, 主要原因可能如下 2 点:

- 1) 所有的 stop_grad 和 not trainable Var, 导致反向路径上并不是所有的 OP 均需要 OP_grad。甚至有些枝干, 因为一个 stop_grad 可以全部剪枝。因此 minimize 需要做优化, 尽量减少不必要的 OP 的 Append。
- 2) 由于 control flow op 以及对应的子 Block 的存在, 导致反向过程多了很多特殊处理。

这里结合 PE 会存在一个问题: minimize 添加了反向到 Program 中, 之后 PE 将 Program 转化成 Graph, 通过 Pass 对 Graph 进行 fuse 优化。假如 mul+add 需要 fuse 成 fc_op, 那么 add_grad+mul_grad 也有必要 fuse 成 fc_op_grad。这里恐怕设计的不太好。

3.3.3 Run 前的补充优化

这部分, 从 API 上, 给用户的感觉是已经开始『Run』了, 然而再真正启动 C++ Executor Run 之前, 我们还会做一些准备工作。这些工作有些属于编译时, 有些介于编译时和运行时之间。姑且全部算作编译时。

3.3.3.1 Prune(剪枝)

这部分没有细看。根据秋良和威行的介绍, 大致背景如下:

在有些场景中, 用户可能定义了一个复杂的网络。但用户最后 Fetch 的只是一个中间数据, 或者提供的 loss 是从网络的一个中间节点开始反向的。那么类似这两种情况, 用户所定义的复杂网络中, 有很多网络分支是完全没必要 Run 的。我们可以提前剪枝掉, 让用户尽早完成训练, 获得期望的数据。

3.3.3.2 Feed、Fetch OP 添加

用户在调用 `exe.run` 时，提供了 `main_program`、`feed`、`fetch_list`，此时的 `main_program` 拥有了整个网络的定义，包括后向定义。但是数据是如何送入网络的呢？用户又是如何观察网络迭代训练的过程数据的呢？

首先，用户是提前预处理好数据，每次通过 `feed`，向 `exe.run` 中传入一个 batch 的真实数据，这个数据已经存储在内存中。那么这些数据需要送入相应的 Var（占位 Var）中，比如我们 Demo 中的占位 Var X 和 Y。

实现方法是，Python 端的 Executor 需要创建一个 Feed Var，这个 Var 有多列（col），每列对应一个占位 Var。之后为每个占位 Var 创建一个 Feed OP，这个 OP 负责将 Feed Var 中相应列中的数据复制到占位 Var 中。这里的复制是指针拷贝，不是内存拷贝。

Fetch 过程类似，也是需要创建一个 Fetch Var 和相应的多个 Fetch OP。Fetch Var 存储用户 Fetch 的所有数据。

Feed Var 是如何拿到真实数据的，Fetch Var 是如何将数据返回给用户的。我们在 1.3.3.4 中简单介绍。

3.3.3.3 Catch 机制

我们知道，在训练过程中，1 个 batch 需要 `exe.run` 一次，总体 `exe.run` 需要 run 非常多次。对于以上的 Feed、Fetch OP 的添加，如果每次 `exe.run` 都执行一次，是不合理的、浪费效率的。

同样的问题还存在于 C++ Executor 的 Run 中，如：

- 1) Context 中将 OPDesc 转 OperatorBase
- 2) Context 中 UnusedVar 的计算
- 3) Scope 中每个 C++ Variable 的 Create

C++ Executor Run 中涉及的两个业务我们后续展开，这里要表述的是，这些逻辑没必要多次执行，浪费效率。

为此，我们引入了 Catch 机制，在首次计算得到 Program、Context、Scope 后，我们在 Python 端 Catch 一份，以后每次用的时候，用 Catch 中的那份，而不是再重复构造。这就是 Catch 机制。

假如后续执行过程中，某些使用发生了改变，如 `fetch_list`。则需要重新计算 Program、Context、Scope。因此我们引入了 `catch_key`，唯一标识一个 Program、Feed、FetchList 的组合。

3.3.3.4 Feed 数据到 Feed Var 中，返回 Fetch 数据

之所以把这部分放到 Catch 机制后面表述，是因为虽然 Feed OP、Fetch OP 可以计算一次，Catch 住，重复使用。但每次 Feed 的数据，Fetch 的数据是不一样的，是不能 Catch 的。

在 C++ Executor Run 之前，Python 端调用 `_feed_data` 函数，将用户提供的 feed（真实数据），分别拷贝到 Feed Var 相应的 col 中（这里也是指针拷贝）。这个拷贝结合 Feed Op 完成，确保 col 于相应的占位 Var 正确对应。

类似的 C++Executor Run 结束后，将 Fetch Var 中的数据整理好返回给用户。

补充一点题外话：我们每次向 Executor 中 Feed 的是一个 Batch 的数据，即 N 个 X 和 N 个 Y 。一个 `exe.run`（即一个前向、反向过程），是执行 1 次还是 N 次呢？答案是 1 次。我们在每个 OP 执行的时候，都是拿着整个 Batch 的数据进行计算，并输出到下一个 Var 中的。

总结：在 C++Run 之前，有两种 Var 是已经准备好了的，包括数据显存和数据值。这两中 Var 分别是 Parameter 和 Feed Var。

3.4 运行时

从 1.3 节，我们了解到，编译时已将用户定义的一个网络模型，转义成 Program。这个 Program 是经过一定优化的，内部使用 BlockDesc、OpDesc、VarDesc 表述的一个『隐晦』的图。之所以隐晦，是因为 OpDesc、VarDesc 都是用 Vector 存储的，OpDesc 中通过 inputs、outputs 隐晦的建立的关系（边）。需要补充说明的是：编译时 Append OP 是根据 OP 的执行先后循序 Append 的，即需要先运行的 OP 先 Append，后运行的 OP 后 Append。

因此，运行时的输入是：

- 1) Program;
- 2) Parameter Var、Feed Var;

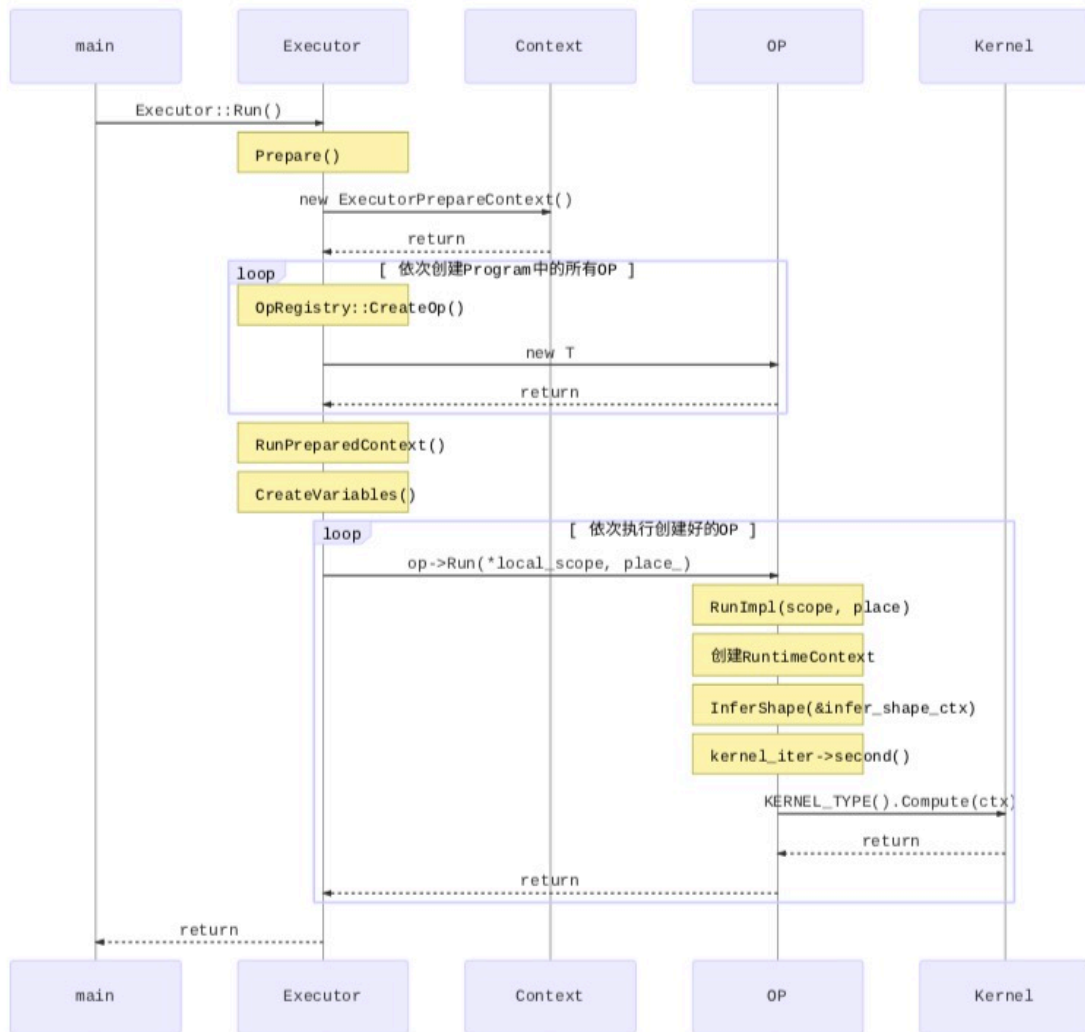
运行时的输出是：

- 1) 更新后的 Parameter（这是我们训练的根本目的）。
- 2) 生成 Fetch Var 提供给用户。

运行时的主要任务有：

- 1) 转换 Program，生成 C++可执行的 OperatorBase、Variable;
- 2) 顺序执行 OP;
- 3) 完成内存清理，也要考虑内存优化。

以下是胡晓光给出的运行时时序图：



下文就每个环境，详细介绍。

3.4.1 Prepare

Prepare 做了两件事：

- 1) OpDesc 到 OperatorBase 的转化
- 2) unused_vars_的构造

OpDesc 到 OperatorBase 的转化非常简单，遍历所有 OpDesc，OpDesc 中包括了创建 OperatorBase 的所有信息。通过 type 在 OpInfoMap 中找到相应的 OpInfo，调用 creator 即可创建。创建后的 OperatorBase，以 Vector 存储在 ExecutorPrepareContext 中。

unused_vars_的类型是 `std::unordered_map<const OperatorBase*, std::vector<std::string>>`。其含义是，当某个 OperatorBase 执行完成后，对应的几个 Var 将不在有其它 OperatorBasse 使用，可以提前释放。

什么样的 var 可以中途释放呢？

- 1) 不能是用户 Fetch 的，因为释放了用户就拿不到了；

- 2) 不能是 persistable 的，这个属性的意思就是不能被优化；
- 3) 只能是 LOD_TENSOR、SELECTED_ROWS、LOD_TENSOR_ARRAY 三中类型中的一种。这个为啥不清楚。
- 4) 在 control flow 的 Block 中，涉及到 control op 要使用的 var 不能被优化。代码中有三个函数在干这方面的事儿，看到的时候别疑惑：
PrepareSafeEagerDeletionOnConditionalOpAndConditionalGradOp、
PrepareSafeEagerDeletionOnWhileOpAndWhileGradOp、
PrepareSafeEagerDeletionOnRecurrentOpAndRecurrentGradOp

构造 unused_vars_ 的算法很容易看懂，遍历一遍 ops，标记每个变量最后一次被那个 ops 使用（inputs、outputs）。之后反向生成 unused_vars_ 即可。

在 Catch 机制下 Prepare 过程是由 Python 端提前调用，将 Context Catch 住的。

3.4.2 RunPreparedContext

RunPrepareContext 首先是在 Scope 中 Create Variable，初始化 Variable。（这个环节再 Catch 机制中，由 Python 主动调用 CreateVariables，之后 RunPrepareContext 就不用 Create 了。）

之后循环、顺序执行 Context 中的 OP，每个 OP 执行完后根据 UnusedVar，释放已经不再使用的 Var 交给 GC。

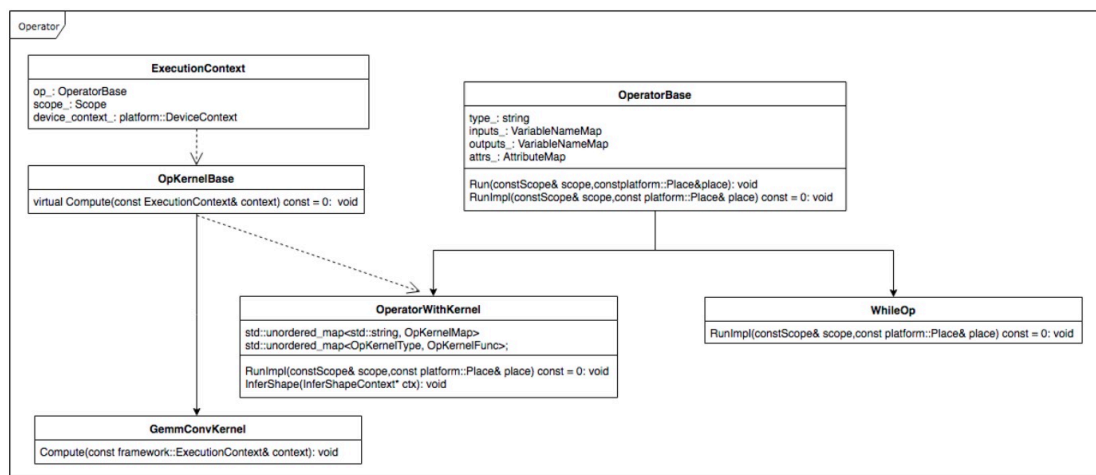
最后视情况释放 Scope。

3.4.3 OperatorBase

Paddle 中所有 OP 都是继承自 OperatorBase，OperatorBase 的核心方法是 Run，Run 函数调用 RunImpl 实际执行计算。

Paddle 的设计理念是可以在多种设备（Place）上运行，加上变量类型的不同。往往一个 OP 在不同情况（Place、变量类型）下的代码实现完全不同。因此引入了 OpKernel 的概念，即 OperatorWithKernel，他继承自 OperatorBase，并重写了 RunImpl 方法。

此外，还有一种控制流 OP，如 WhileOp，是直接继承自 OperatorBase 重写 RunImpl 的，我们在下节介绍。以下是胡晓光给出的类图：



3.4.4 OperatorWithKernel OP Run

3.4.4.1 Kernel 的介绍

Kernel 的注册于 OP 的注册类似。这里不再介绍。

Kernel 在 Paddle 中的管理代码片段截取如下：

```

std::unordered_map<std::string, OpKernelMap> g_all_op_kernels;
using OpKernelMap = std::unordered_map<OpKernelType, OpKernelFunc, OpKernelType::Hash>;

OpKernelType(proto::VarType::Type data_type, platform::Place place,
             DataLayout data_layout = DataLayout::kAnyLayout,
             LibraryType library_type = LibraryType::kPlain,
             int customized_type_value = kDefaultCustomizedTypeValue)

OperatorWithKernel::AllOpKernels()[op_type][key] = func;

using OpKernelFunc = std::function<void(const ExecutionContext&)>;

template <typename DeviceContext, typename T>
class MulKernel : public framework::OpKernel<T> {
public:
    void Compute(const framework::ExecutionContext& context) const override {

```

通过[op_tpye][key]，可以看出，Kernel 通过 op_type(std::string)+key(OpKernelType)两级索引管理 OpKernelFunc。而 key 又由很多信息组成，不同信息组成的 key 不同，就需要不同的 OpKernelFunc 支持。

而 func 是指向一个 Kernel Compute 函数的函数指针。Compute 是 Kernel 执行计算的主体，传入的是 ExecutionContext 参数。

3.4.4.2 RuntimeContext

RuntimeContext 中存储了 OP 运算时需要用到的 Variable 指针，包括 inputs 和 outputs，使用 VariableValueMap = std::map<std::string, std::vector<Variable*>> 方式存储。

3.4.4.3 OP 的 Run

Step1:RuntimeContext 的构造

OperatorWithKernel 中存储了其计算的输入、输出的变量名，在进行计算前，他需要找到『变量值』，即 Scope 中的 Variable。OperatorWithKernel 通过构造 RuntimeContext 将『变量值』存到 RuntimeContext 中。

为了提升性能，OperatorWithKernel 中也有一个小的 Catch 功能，可以通过设置 Arrt kEnableCacheRuntimeContext 将该功能打开。此功能若打开，OperatorWithKernel 对象中将持有 RuntimeContext，进而不用每次都构造 RuntimeContext。

Step2: DeviceContext 的准备

DeviceContext 是提前初始化好的，不同的硬件所需初始化的工作不同，初始化后需要保留的变量不同。因此使用不同的 DeviceContext 负责初始化的执行和变量的存储。OP 在 Run 时根据 Place 到 DeviceContextPool 中获得对应的 DeviceContext。

题外话：上面已经提及好多个 XXXContext，都有哪些 Context 呢？都是干嘛用的呢？胡晓光做了总结：

Context用来表示执行特定计算所需要的上下文信息。Context通常为调用某一个函数所需的参数组合而成的结构体，比如RuntimeInferShapeContext为执行op->InferShape所需参数的集合。

Context类型	生命周期	用途
DeviceContext	执行Init后的整个程序周期 DeviceContextPool::Init()	CPU场景eigen设备初始化；GPU场景cuda, cudnn设备初始化。 DeviceContextPool::Instance().Get(place_)
ExecutorPrepareContext	Executor::Run()	创建执行Run所需的op，创建unused_var
RuntimeContext	OperatorWithKernel::RunImpl()	根据当前op的输入和输出名称，找到scope中对应的变量指针，存放于当前context中
ExecutionContext	OperatorWithKernel::RunImpl()	GetExpectedKernelType传入数据类型和place；执行kernel->Compute所需的上下文信息，包括op, scope, device_ctx, runtime_ctx。
InferVarTypeContext	OpDesc::InferVarType(BlockDesc *block)	编译时调用，python组网时调用，包含当前OpDesc引用和BlockDesc引用
RuntimeInferVarTypeContext		在动态图时使用
InferShapeContext		定义接口，参见子类
CompileTimeInferShapeContext	OpDesc::InferShape(const BlockDesc &block)	编译时调用，python组网时调用，包含当前OpDesc引用和BlockDesc引用
RuntimeInferShapeContext	OperatorWithKernel::RunImpl()	执行当前op->InferShape所需参数的集合。包括当前op指针，执行scope, runtime_context。

Step3: Kernel 的选择

3.4.3.1 中我们介绍了 Kernel 的管理。Kernel 的选择，其实就是构造 op_type 和 key，然后到 AllOpKernels()里去查找的过程。在 ChooseKernel 函数中实现。最终将 Kernel 的函数指针放到了 kernel_func_成员变量中。

Step4: 数据转换

为啥要数据转换？对于 Kernel 的一些输入变量，如果其变量格式和 Kernel 可处理的变量格式不一样，会导致 Kernel 无法处理。这里所说的变量主要指 LoDTensor 和 SelectedRows 两种类型。格式不同的原因包括（NeedTransform 函数中实现）：

- 1) 其所在的 Place 的类型不同
- 2) 基础数据 DataType（如 BOOL、INT16、FLOAT）
- 3) DataLayout（如 NHWC、NCHW 等）不同

因此需要进行数据格式转换。

这里有个问题，转换好的、新的 Variable（trans_var），并没有更新到 RuntimeContext 中。而后续计算使用 Variable 时，都是使用的 RuntimeContext 中的 Variable。那数据不是白转换了吗？——这个是在 PrepareData 函数中，每次数据转换完，他替换了 ctx 得 input_vars[i]。因此，这只是更改了 RuntimeContext 中的 Variable 的指针。

而原来的 Scope 中的 Variable 还是指向数据转换之前的个 Tensor 的。
TransferInplaceVarsBack 函数可以把 Scope 中的数据格式也给换成新的。

Step5: 构造 ExecutionContext, 并执行 Kernel 的 Compute 函数

ExecutionContext 可以认为是一系列参数的载体, 这些参数, Compute 将来可能会用到。

执行 Kernel 就是调用已经选好的 Kernel 的函数指针, 去执行
其余还有很多开关涉及的逻辑, 在这里不再介绍。

3.4.4.4 Kernel 的 Compute

Kernel 的 Compute 负责更底层的负责 OP 的实际计算逻辑, 主要步骤大概有:

- 1) 从 Context 中取得 inputs 的 Tensor 指针;
- 2) 为 output Variable 的 Tensor 分配真实的内存 (可能是内存也可能是显存);
- 3) 调用更底层的算子执行。

Kernel Compute 还不是最底层的实际执行逻辑, 还有更底层的。。。但于框架调度关系已经不大。

3.4.5 Control Flow OP Run 的细节

Paddle 中的控制流也是 OP, 他不负责实际数据的计算, 因此不需要 Kernel。这种 OP 直接继承自 OperatorBase, 并重写 RunImpl 函数。并在 RunImpl 函数中实现控制流的主体逻辑。

我们仍以 3.3.1.4 中的 cond 举例介绍, 他的 C++ OP 是 ConditionalOp。

```
def true_func():
    return layers.fill_constant(
        shape=[1, 2], dtype='int32', value=1), layers.fill_constant(
            shape=[2, 3], dtype='bool', value=True)

def false_func():
    return layers.fill_constant(
        shape=[3, 4], dtype='float32', value=3), layers.fill_constant(
            shape=[4, 5], dtype='int64', value=2)

main_program = Program()
startup_program = Program()
with program_guard(main_program, startup_program):
    x = layers.fill_constant(shape=[1], dtype='float32', value=0.1)
    y = layers.fill_constant(shape=[1], dtype='float32', value=0.23)
    pred = layers.less_than(x, y)
    out = layers.cond(pred, true_func, false_func)
```

首先, C++ Executor 的 Run 的功能是对一个 Block 进行 Run。因此 Python 调用时, 因为要 Run Program 主体, 因此传入的 Block ID 为 0。那么什么时候 Run 其它的 Block 呢, 由谁发起的呢?

在 Block 0 中, Control Flow OP 的定义。比如 Cond 的 Demo。Paddle 为 true_func 和 false_func 分别创建了一个 Block, 也同时向 Block 0 中加入了两个 ConditionalBlockOp, 每个 OP 指向其中一个 Block。但 C++ Executor 运行 Block 0 时, 根据 pred Var 的值, 分别运行这两个 ConditionalBlockOp, 其中一个会执行 need_run。

在 need_run 中，为该 Block 重新准备了 Scope、Executor，然后让这个 Executor 去 Run 这个子 Block。代码中的 skip_vars，就是 3.4.1 节中讲到的，不能被优化的 Var。

值得注意的是，Control Flow OP Run 的过程中，需要重新构造 OperatorBase、UnusedVar、Variable (Scope)，性能上是有些欠理想的地方的！

3.5 其它

3.5.1 OP 的管理

Paddle 需要不断新增 OP，支持各种计算。因此 Paddle 必须定义统一的调用接口，和 OP 管理方案。统一调用接口即 OperatorBase。OP 的管理则包括了 OP 的开发、OP 的注册和创建使用。

OP 的开发可参考官方文档：

https://www.paddlepaddle.org.cn/documentation/docs/zh/advanced_guide/addon_development/new_op/new_op.html

OP 的注册在陈威行的《Paddle Fluid 框架执行逻辑梳理》讲的比较细。本文做点文字表述：

- 1) 所有的 OP 有 OpInfoMap 统一管理，他是单例实现，核心成员是 std::unordered_map<std::string, OpInfo> map_。注册时向其插入 OpInfo。需要创建时，查找 OpInfo，完成相应的创建、Check 等操作。
- 2) OpInfo 包括了 OP 构造环节所需的所有类、函数指针：

```
class OpInfo {
public:
    OpCreator creator_;
    GradOpMakerFN grad_op_maker_;
    proto::OpProto* proto_{nullptr};
    OpAttrChecker* checker_{nullptr};
    InferVarTypeFN infer_var_type_;
    InferShapeFN infer_shape_;
    InferInplaceOpFN infer_inplace_;
    InferNoNeedBufferVarsFN infer_no_need_buffer_vars_;
    DygraphGradOpMakerFN dygraph_grad_op_maker_;
```

- 3) OP 通过 REGISTER_OPERATOR 进行注册，一次性将所有『构造执行实体』传入。OP、OP_grad 需要注册两次。

```
REGISTER_OPERATOR(mul, ops::MulOp, ops::MulOpMaker, ops::MulOpInferVarType,
                  ops::MulOpGradMaker<paddle::framework::OpDesc>,
                  ops::MulOpGradMaker<paddle::imperative::OpBase>);

REGISTER_OPERATOR(mul_grad, ops::MulGradOp,
                  ops::MulDoubleGradMaker<paddle::framework::OpDesc>,
                  ops::MulDoubleGradMaker<paddle::imperative::OpBase>);
```

- 4) REGISTER_OPERATOR 实例化了一个 OperatorRegistrar，OperatorRegistrar 用到了 C++ 中的模板元编程，模板可以被递归调用，在递归过程中，完成 OpInfo 中一个成员变量的初始化。

模板元编程，使得 OperatorRegistrarRecursive 被循环构造，每次构造根据模板参数的不同，OpInfoFiller<T> fill 会选择不同的 Filler，每个 Filler 负责了 OpInfo 一个成员变量的初始化。

```
template <size_t I, typename... ARGS>
class OperatorRegistrarRecursive<I, false, ARGS...> {
public:
    using T = typename std::tuple_element<I, std::tuple<ARGS...>>::type;
    OperatorRegistrarRecursive(const char* op_type, OpInfo* info) {
        OpInfoFiller<T> fill;
        fill(op_type, info);
        constexpr auto size = sizeof...(ARGS);
        OperatorRegistrarRecursive<I + 1, I + 1 == size, ARGS...> reg(op_type,
                                                                    info);
        (void)(reg);
    }
};

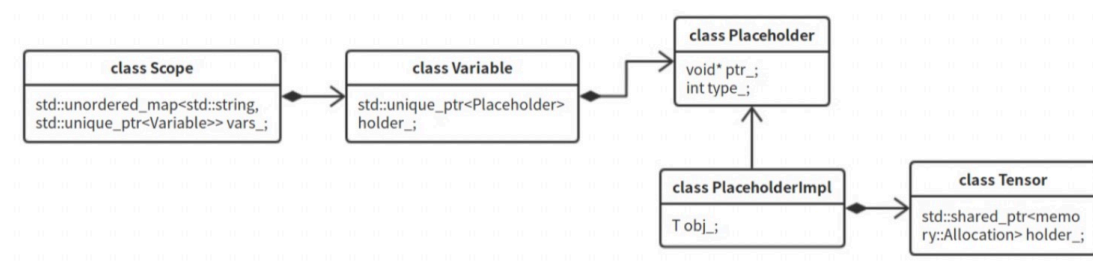
template <typename T>
struct OpInfoFiller<T, kOperator> {
    void operator()(const char* op_type, OpInfo* info) const {
        PADDLE_ENFORCE_EQ(info->creator_, nullptr,
                          platform::errors::AlreadyExists(
                              "OpCreator of %s has been registered", op_type));
        info->creator_ = [](const std::string& type, const VariableNameMap& inputs,
                          const VariableNameMap& outputs,
                          const AttributeMap& attrs) {
            return new T(type, inputs, outputs, attrs);
        };
    };
};
```

OP 的创建，则是在 C++ Executor Run 时，根据 Desc 中的信息，通过 OpInfoMap 找到相应的 OpInfo 创建 OP。

OpInfo 中出了创建 OP 的 creator，还有很多和构造相关的类，他们都是干嘛用的，尚未深入学习。

3.5.2 Scope、Variable、Tensor、mem

以下是相关类的类图，对应相应的代码一目了然。Placeholder、PlaceholderImpl 也很简单，不再展开介绍。



Scope 是运行时的一个概念，可以认为他是 Variable 的『容器』，他通过 unique_ptr 方式持有一个 Variable 的指针。若 Scope 析构，相应的 Variable 也会被释放。注意，在 RuntimeContext 中持有的是 Variable*，而这个指针是从 Scope 中 unique_ptr 中 Get 的，因此不需要释放。但，如果 Scope 析构了，RuntimeContext 中的这个指针就是野指针了。

因此，Scope 是用于管理 Variable 用的，如果有一些临时 Variable 在某些特定环节结束后即可释放。我们可以 Scope.new_scope()。这样，在父 Scope 下创建一个子 Scope。然后

将临时 Variable 放到子 Scope 中。当子 Scope 释放时，这些临时 Variable 也就跟着释放了。因此，Scope 用于管理 Variable 释放，很好用。

内存分配。Scope 中的 Variable 是在 Executor Run 之前，统一 Create 的，这时候就创建了 PlaceholderImpl 和 Tensor。但是！这个时候的 Tensor 是空的（holder_为 NULL）。Tensor 的实际内存得到 OpKernel Compute 时才会分配。

注意，以上说法有个特例，那就是 FeedVar。这个 Variable，在 Executor Run 之前就放到 Scope 中了，而且将 Tensor 的 holder_也赋值了。

还有值得分析的是，Tensor 中的 holder_是 share_ptr 类型的。在 Paddle 中经常存在多个 Tensor 贡献一块内存数据的情况。因为 Tensor holder_所持有的数据是大量的，内存申请、内存复制都非常耗时，也浪费内存空间。因此代码中有很多地方调用 ShareDataWith 函数共享内存。其实，Scope、Variable、Tensor 都是『管理』内存的，而不真实持有内存，可以说都是索引。只有到 holder_才真实持有内存。

内存释放：Scope 一般都是根据业务需要，其管理的 Variable 不再使用了，就会统一释放 Scope。Scope 释放的时候，一定释放了 Variable 和 Tensor。但是，只有 Tensor 的 holder_是 shared_ptr,因此需要引用计数归零时，才会释放真实内存（holder_）。

3.5.3 gc

gc(Garbage Collection)，用于内存的回收、清理。在 Executor 中的内存回收比较简单。3.4.3 的 Prepare 函数中，构造了一个 UnusedVar，描述了这个 Variable 在某个 OP 之后就无人再用了，可以回收了。那么在某个 OP 执行之后，我们就可以提前释放其对应的 UnusedVar。这里所谓的释放 Var 并不是释放 Variable，而是释放 Variable 对应的 Tensor 中的 holder_。因为 holder_中的内存才是占用较大空间的内存。

gc 的意义。为什么我们要在 Executor 运行过程中清理内存的，清理内存也是要消耗 CPU 的计算资源的，那不是拖满了 Executor 的计算速度吗？——Executor 每次只训练一个 Batch，如果每次训练的 Batch 大，那么需要训练的 Batch 的总个数就会少，循环执行 exe.run 的次数就会少。这样可以大大加快训练时间。但是，如果 Batch 太大了，内存是装不下的。如果我们能够优化内存的使用，可以让每次 Run 的 Batch 增大，就可以提升训练速度。这是内存回收的意义所在。

Executor 中 gc 的执行过程：

- 1) 声明一个 `std::deque<std::shared_ptr<memory::Allocation>>` `garbages`，将需要释放的 `holder_(share_ptr<memory::Allocation>)`,move 到 `garbages` 中。这样，Tensor 的 `holder_`就位 `nullptr` 了。
- 2) 通过 `gc->Add(std::move(garbages))`;将 `garbages` 再 move 到 `gc` 的 Container 中，这个 Container 其实是 `garbages` 的引用。
- 3) 再通过 `auto *container = new Container(std::move(objs))`，new 一个指针类型的 Container。

- 4) 通过 `ClearCallback([container] { delete container; })`，将指针类型的 `container` 交给 `gc` 去 `delete`。当 `container` `delete` 时，里面的 `shared_ptr` 将跟随释放，其引用计数将减 1。

3.5.4 pybind11

`pybind11` 是众多 Python 调用 C++ 的方法之一。`pybind11` 使用比较简单，对 C++11 的支持也很好。Paddle 使用 `pybind11` 完成 Python 调用 C++ 内核。

Paddle 的 C++ 内核会被编译成 `core.so`，Python 端可通过 `core.XXX` 接口进行调用。

3.5.5 Var 的唯一取名

`unique_name.py` 定义了一个字典，在 `UniqueNameGenerator` 中。记录了每个 `name` 被命名的次数 `idx`，首次使用 `id index` 为 0。之后每次 `name` 发生重复，则一次为 1、2、3…。其核心函数为 `generate`。

```
tmp = self.ids[key]
self.ids[key] += 1
return self.prefix + "_" + str(tmp)
```

占位 `Var` 的取名以用户命名为准，内部不再做加工。若用户定义了两个变量，`name` 重复。则框架内部只有一个 `Var`。而用户态的两个 `Var` 均指向这一个框架内的 1 一个 `Var`。若用户没有提供 `name`，则框架为其创建唯一的一个 `name`。

`layers` 下的函数均有可能重复使用，如 `fc`、`mean` 等。这类函数均有一个唯一函数名 (`name`)。如 `fc_0`、`fc_1`、`mean_0` 等，由 `LayerHelper+unique_name.py` 共同完成。

`fc` 内部声明的临时变量、`Parameter` 的 `name` 均以 `fc` 的唯一函数名+“.”为前缀（如 `fc_0.`），以其自身名字为后缀（如 `tmp`、`w`、`b`），并通过 `unique_name.py` 为其后缀添加唯一 `ids`。如：

```
suffix = 'b' if is_bias else 'w'
if attr.name is None:
    attr.name = unique_name.generate("." + self.name + suffix)
```

```
return self.main_program.current_block().create_var(
    name=unique_name.generate_with_ignorable_key("." + self.name + 'tmp'),
```

反向 `Var` 的取名，根据其前向 `VarName`，加上“@GRAD”即可。如 `fc_0.tmp_0` 对应的反向 `Var` 名字为 `fc_0.tmp_0@GRAD`。其实现在 `_append_grad_suffix` 函数中。

而 `OP` 是无状态的，因此 `OP` 不需要为其取名。其输入+输出+type+attr 已经能唯一描述一个计算。

3.5.6 LodTensor&SelectedRows

`LodTensor` 和 `SelectedRows` 都是压缩后的 `Tensor`。

对于一些每行长短不一致，而且长度差异很大的 `Tensor`。如果我们将所有短的行全部不填成最长行的长度。一方面占用大量的显存。另一方面也增大了计算量。这时候，我们

使用 LodTensor。LoDTensor 将长度不一致的维度拼接为一个大的维度，并引入了一个索引数据结构 (LoD) 来将张量分割成序列。具体可参考：
https://www.paddlepaddle.org.cn/documentation/docs/zh/beginners_guide/basic_concept/lod_tensor.html

对于 Tensor 中有大量行全部为 0 的情况，我们可以记录这个 Tensor 一共有多少行，然后把有非 0 数据的行存下来（包括行号），全部为 0 的行不再存储。这样可以大大压缩内存使用量。

3.5.7 Program Graph 的打印

Paddle 生成的 Program 中 Var、OP 数量很大，是一个非常复杂的 Graph。如果靠短点调试或者日志打印。以我们大脑的能力很难想象 Program 中 Graph 的长相。Paddle 中有很好的工具可以将其转换为 Graph 图片输出。

Program 的图片打印可以参考：

https://github.com/wzzju/wzzju.github.io/blob/master/assets/tools/paddle_model_visual.py

PE 中 Pass 后的 Graph 可参考以下链接中的 debug_graphviz_path 的使用：

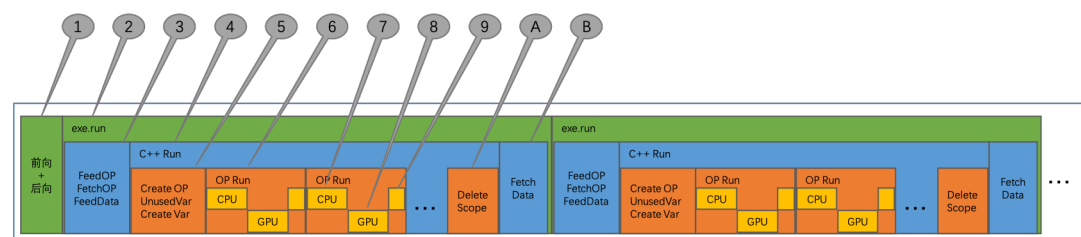
https://www.paddlepaddle.org.cn/documentation/docs/zh/api_cn/fluid_cn/BuildStrategy_cn.html#buildstrategy

他们生成的 Graph 的格式为 Graphviz，可以网上查询 Graphviz 如何使用。转换成 png 的命令可参考：

`dot graph_original_graph -T png -o original.png`

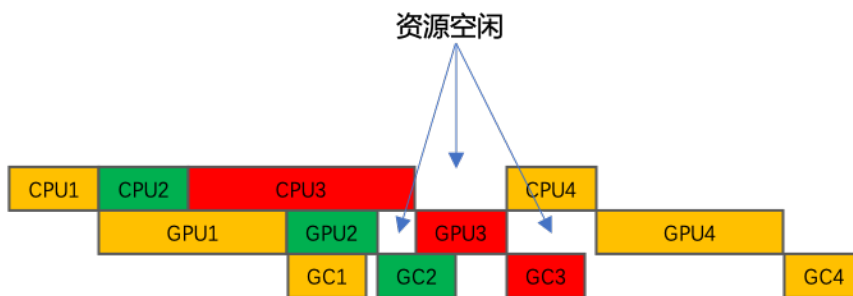
3.6 从 Executor 看 PE 的性能优化

Executor 的执行可以概括为下图：



注意，图中对给人的感觉 OP 的 CPU、GPU 是串行交替执行的。事实并非如此，

CPU、GPU 之间是一定程度上并行的：



以下先对 CPU、GPU 的并行进行一个解释：

上图假设有 4 个 OP（OP1，OP2，OP3，OP4）那么 CPU、GPU、GC 的执行有如下规律：

- 1) GC 也由 CPU 执行，但是单独线程。因此环境准备与 GC 之间是可以并发的；
- 2) GC1 必须等到 GPU1 执行完才能执行，GPU1 必须在 CPU1 环境准备好后，由 CPU1 触发；
- 3) GC1 的销毁任务，是 CPU1 提前提供的，GC1 阻塞等到 GPU1 结束后，开始销毁；
- 4) CPU1 结束后，大多数情况可以直接启动 CPU2 的环境准备工作。这需要有前提，就是 OP 内没有特定的 `cudaStreamSynchronize wait` 动作。如果有必要的 `cudaStreamSynchronize` 则会造成 CPU3~CPU4 间的 CPU 资源浪费，进而势必存在 GPU3~GPU4 间的资源浪费。
- 5) 如果 CPU3 的环境准备时间太长，导致 GPU 所有任务都完成了，那么也会造成 GPU 的无任务状态，造成 GPU2~GPU3 间的资源浪费。

以下对 Executor 的执行过程，以及存在的可能的优化点进行分析：

- 1) Program Desc 的生成，这部分主要是『原样』描述前向网络，自动生成后向网络。

优化点 1：而后向网络中有些路径是没有必要的，如 `stop_grad`，因此可以做剪枝。

2) 循环执行 exe.run，每次执行一个 Batch。假如 1 个数据集由 N 个 Batch，一共训练 M 轮，则 exe.run 需执行 $N \times M$ 次。

3) 补全 Feed、Fetch OP 和 Var，并将训练数据集数据放入 Var 中。

优化点 2: 如果用户 Fetch 的变量，不需要将整个网络跑完即可提供给用户。那么网络中可能存在『不需要』的分支。我们可以进行 Prune 剪枝。

优化点 3: 除了 stop_grad、Prune，在 C++ Executor Run 之前还有很多能做的优化。因为 Program Desc 是一个接近于用户表述的网络的『原始』描述。在 PE 中，Paddle 将 Program 转换成 Graph，通过 **BuildStrategy**（一系列的 **Pass**），对 Graph 进行优化。这样，可以在 C++ 实际 Run 之前，进行 Desc 级的大幅优化。

优化点 4: 如果每次都进行补全 Feed、Fetch OP 和 Var 的操作，属于重复计算。我们可以将 Program **『Catch』** 住，减少这部分重复计算。

4) C++ 的 Executor Run，主要包括了环境的准备、所有 OP 的 Run、Scope 的释放。

5) OP Run 之前的环境准备。包括 OP 的 Create、UnusedVar 的构造、Scope 中 Variable 的 Create。

优化点 4: OP 的 Create 和 UnusedVar 的构造，组成了 ExecutorPrepareContext，如果每次 exe.run 都重复的执行这两个构造，也是不需要的。可以将 Context **『Catch』** 住。

优化点 4: Scope 中的 Variable 的构造也不需要重复构造，可以将 Scope **『Catch』** 住。

6) 一次 C++ Run 将有 N 个 OP 循序执行。每个 OP 执行均包括了计算前的准备、计算、内存的回收。计算前的准备和内存的回收，由 CPU 完成。计算往往由 GPU 完成。

7) CPU 完成计算前的准备工作。3.4 节中的内容几乎全部属于计算前的准备。

8) GPU 完成计算。在实际训练中，绝大多数使用 GPU 完成实际计算任务。因涉及 GPU 编程，是 Kernel 更底层的逻辑，本文中没有介绍。

9) 内存的回收，指的是 GC 由 CPU 完成。

优化点 5: 训练过程是期望 CPU、GPU 都 100% 负荷工作的，即尽量充分利用计算资源。因此 Executor、PE 都应考虑，如何减少 CPU、GPU 的空闲间隙，尤其是 GPU 的空闲间隙。

优化点 6: CPU 是多核的，GPU 跟有大量的计算单元。一个 OP 可能无法完全让 CPU、GPU 资源利用率达到 100%。同时，很多时候，存在多个 OP 均已具备 Run 的条件（Run 的条件，即 inputs 的 Var 均已计算得出）。这时候，可以让多个 OP 并行 Run，进而充分利用 CPU 和 GPU。PE 的 GraphExecutor 中实现了类似功能。

优化点 7: 一台服务器，可以有 8 个 GPU。可以将 1 个 Batch 分成 8 份，让 8 个 GPU 同时计算，然后将 8 个 GPU 的 Parameter@Grad 全部收集、再分发到各卡去 sgd。这就是 PE 的一机多卡。

优化点 8: CPU 7)、9) 两步，最频繁的操作就是显存的申请和释放。如果能够加速显存的申请和释放，那么 CPU 的计算效率将大幅提升。同时，如果能够尽早回收显存（GC），可以让 Batch 更大。这就是显存优化。

优化点 9: 部分训练由于计算量、数据量过大，单机多卡无法满足需求。我们需要调配多台计算同时计算。这就需要分布式控制。

A) 将 Scope 中的 Variable 释放。

B) 计算完后，将用户 Fetch 的结果返回。

性能疑问：

- a) Pass 的优化有一个问题，他是对整个前向+反向的 Graph 进行优化。那么比如前向有个 fuse，一定对应个反向的 fuse。前向生成完就进行 pass，再生成后向，将更合理一些。
- b) 控制流是没有 catch 的，需要在控制流 OP 中进行子 Block 的 OP Create、UnusedVar 构造、Scope Var Create。
- c) 1 个 OP 就将 GPU 占满，其它排队。那么再并行计算就没意义了。CUDA 的 Grid、Block、Thread 是否需要控制？
- d) 8 个 GPU 并行，reduce all 必须 8 个都计算完，会不会出现 7 个等 1 个的情况？